Customer Data Engineering Pipeline

SQL Server Incremental ETL, Data Quality, Audit Logging, and Local PySpark Analytics

Author: Ramil Khalilli (ramil.khalil.2001@gmail.com)
Project: Customer Data Engineering Pipeline
Technologies: SQL Server 2022 · T-SQL · PySpark 4.0.4 · Python · Parquet · GitHub
Repository: `customer-data-engineering-pipeline`

---

1. Executive Summary

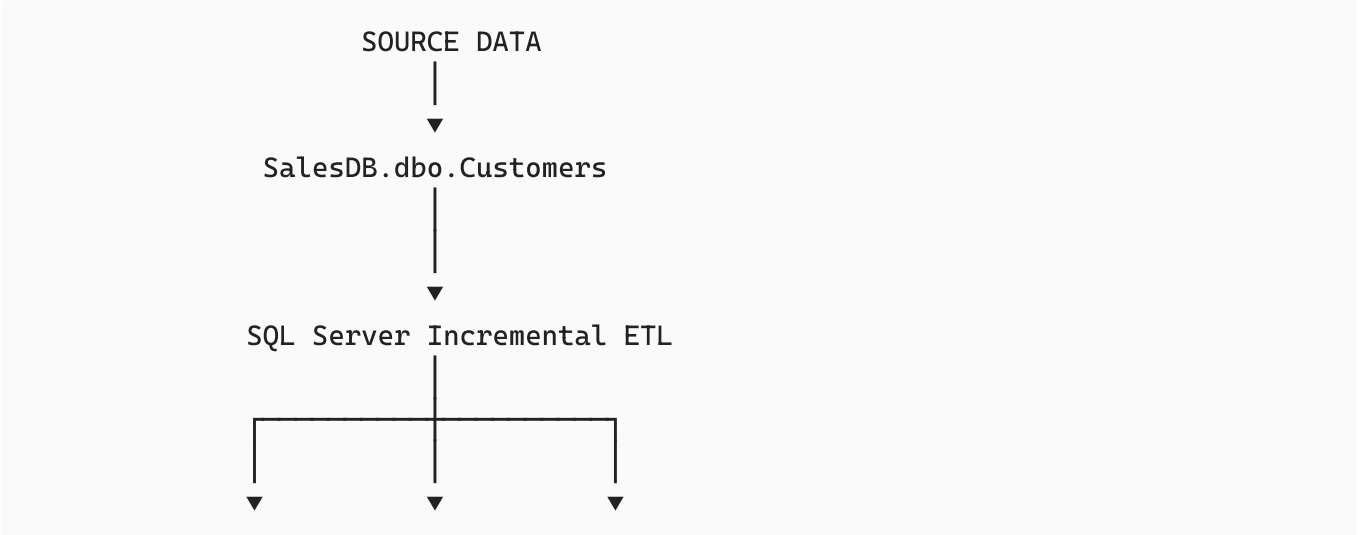
This project is an end-to-end **customer data engineering pipeline** designed to demonstrate practical skills in SQL Server, incremental data processing, data quality, audit logging, transaction management, error handling, and PySpark-based analytical processing.

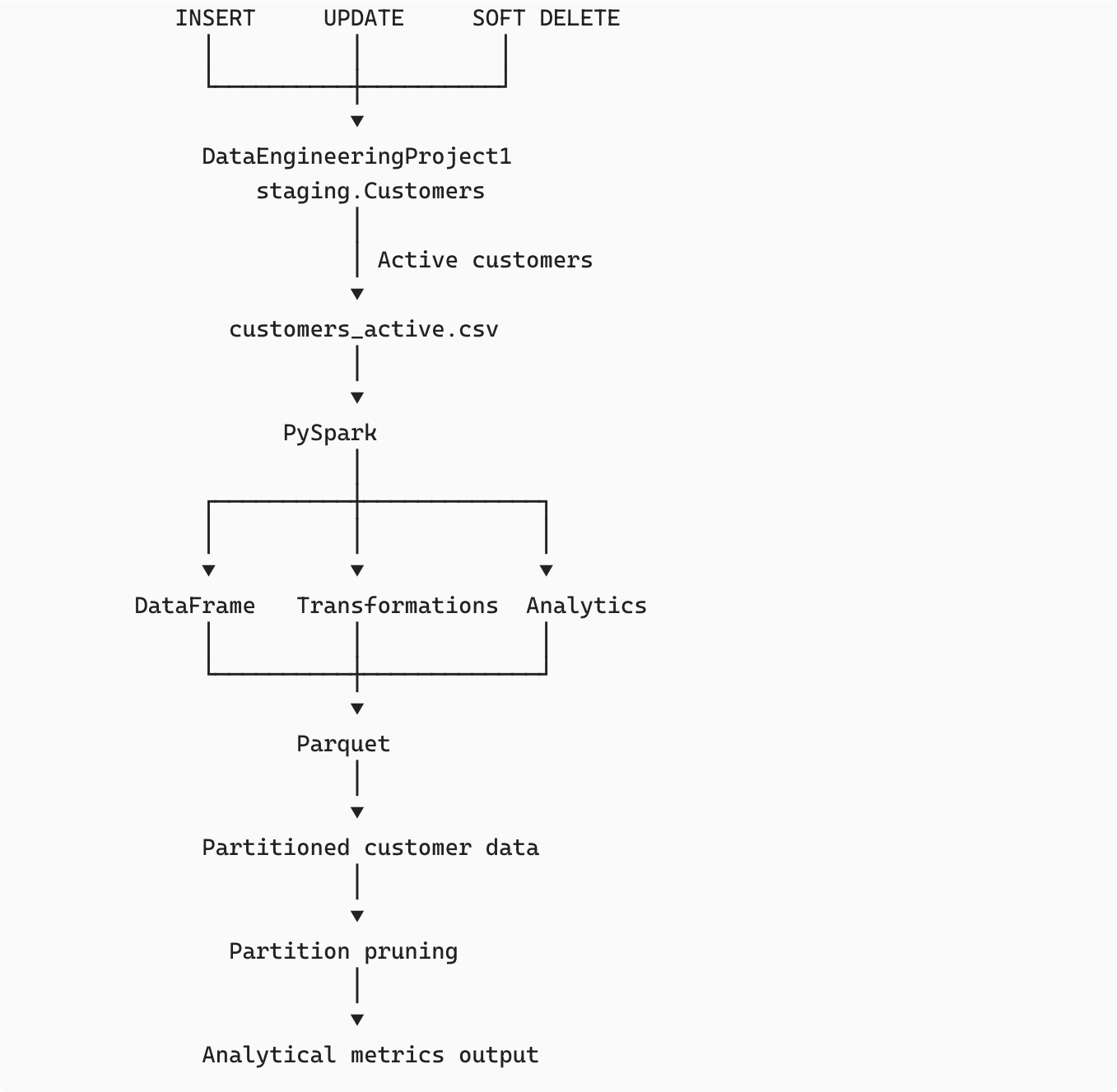
The project was deliberately designed as a **local data engineering environment** rather than relying on paid cloud infrastructure. SQL Server is used as the primary relational data platform and operational staging layer, while PySpark is used locally to demonstrate modern distributed-data processing concepts and analytical transformations.

The central idea of the project is:

> **Build a reliable SQL Server incremental pipeline that maintains a current customer dataset, then use PySpark to transform that curated dataset into analytical outputs.**

The resulting architecture is:





The SQL Server component demonstrates how to build an incremental pipeline capable of identifying:

- new customers,
- changed customers,
- customers removed from the source,
- and customers that have already been processed.

The pipeline uses a **SHA-256 row hash** to detect changes efficiently across multiple customer attributes.

It also includes:

- pipeline execution logging,
- run IDs,
- start and end timestamps,
- success/failure status,
- inserted/updated/deleted row counts,
- error-message logging,
- transactions,
- rollback handling,
- duplicate checks,
- source-to-target validation,
- and soft-delete logic.

The PySpark component then takes the active customer dataset and demonstrates:

- Spark DataFrames,
- schema inference,
- DataFrame transformations,
- filtering,
- column derivation,
- aggregation,
- lazy evaluation,
- physical execution plans,
- Parquet storage,
- partitioning,
- partition pruning,
- analytical segmentation,
- and data-quality reconciliation.

At the end of the project, the active customer population contained **18,484 customers**.

PySpark analysis identified:

- **7,819 customers** in the United States,
- **3,591** in Australia,
- **1,913** in the United Kingdom,
- **1,810** in France,
- **1,780** in Germany,
- **1,571** in Canada.

An income segmentation exercise identified:

- **16,286 Standard Income customers**
- **2,198 High Income customers**

The project therefore demonstrates not only the ability to write SQL and PySpark code, but also the ability to think about **data lifecycle, reliability, validation, storage, performance, and analytical usability as one connected engineering problem**.

---

2. Project Objectives

The primary objective was to build a realistic customer data pipeline that demonstrates the skills expected from a junior or aspiring data engineer.

The project focused on five major areas.

2.1 Incremental Data Processing

Instead of repeatedly replacing the entire customer dataset, the pipeline identifies the differences between the source and the staging layer.

The pipeline handles:

- new records,
- modified records,
- deleted records.

This demonstrates understanding of incremental ETL rather than simple full-table loading.

2.2 Data Change Detection

A SHA-256 hash is generated from the relevant customer attributes.

The hash allows the pipeline to compare the current source version of a customer against the stored staging version.

Conceptually:

```
Customer attributes
        │
        ▼
   SHA-256 hash
        │
```

```
        ▼
    row_hash
```

If the hashes differ, the customer is treated as changed.

This avoids having to compare every individual business column separately during the update decision.

2.3 Pipeline Reliability

The pipeline was designed not only to move data but also to provide evidence about whether the process succeeded.

A dedicated `pipeline_log` table records information such as:

- pipeline name,
- run ID,
- start time,
- end time,
- status,
- inserted rows,
- updated rows,
- deleted rows,
- error message.

This provides an audit trail of pipeline executions.

2.4 Data Quality and Validation

The pipeline includes validation queries to check:

- total rows,
- active rows,
- deleted rows,
- duplicate CustomerIDs,
- active records missing from the source,
- source records missing from the staging layer.

This is important because a data pipeline should not be judged only by whether it executes without producing a SQL error.

It should also be able to demonstrate that the resulting data is correct.

2.5 Analytical Processing with PySpark

The second major objective was to demonstrate PySpark without introducing unnecessary paid cloud infrastructure.

The SQL Server pipeline acts as the upstream data-processing layer, while PySpark is used for analytical transformations.

This provides practical experience with:

- Spark DataFrames,
- Spark transformations,
- aggregations,
- execution plans,
- Parquet,
- partitioning,
- partition pruning,
- and analytical data products.

---

3. Technology Stack

The project uses the following technologies:

Technology	Purpose
Microsoft SQL Server 2022	Relational database and ETL platform
T-SQL	Incremental ETL, validation, logging and transaction management
SQL Server Management Studio	Development and database administration
Python	PySpark development environment
PySpark 4.0.4	Analytical data processing
Google Colab	Local-notebook-style environment for PySpark development

Technology	Purpose
CSV	Data transfer between SQL Server and the Spark environment
Parquet	Analytical columnar storage
GitHub	Version control and project presentation
Markdown	Technical documentation

The SQL Server instance used during development was Microsoft SQL Server 2022 Developer Edition.

---

4. Why the Project Uses Local Processing Instead of Paid Cloud Infrastructure

An early consideration for the project was the use of Microsoft Azure and potentially Azure Data Factory.

However, the project was intentionally redesigned to avoid dependency on paid cloud services.

The objective was not to demonstrate that a particular cloud platform can be configured. The objective was to demonstrate **data engineering capability**.

Therefore, the final architecture focuses on:

```
SQL Server
+
T-SQL
+
Python / PySpark
+
Parquet
+
GitHub
```

This approach has several advantages for this project.

First, the complete pipeline can be reproduced without requiring a paid cloud subscription.

Second, the project remains focused on engineering concepts rather than cloud account configuration.

Third, PySpark can still be demonstrated meaningfully because Spark supports local execution.

The Spark session was configured with:

```
Master: local[*]
```

which means Spark was executed locally using the available CPU cores.

This allowed the project to demonstrate genuine Spark concepts without pretending that a cloud cluster was necessary for the dataset being processed.

---

5. Source Data

The original customer data is maintained in:

```
SalesDB.dbo.Customers
```

The project uses customer attributes including:

- CustomerID
- CustomerName
- BirthDate
- Gender
- MaritalStatus
- TotalChildren
- HouseOwnerFlag
- NumberCarsOwned
- Education
- Occupation
- Continent
- Country
- State
- City
- YearlyIncome
- DateFirstPurchase

The source dataset represents the upstream customer population from which the incremental pipeline operates.

6. SQL Server Data Engineering Layer

The first major component of the project is the SQL Server ETL pipeline.

The target staging table is:

```
DataEngineeringProject1.staging.Customers
```

The staging table represents the curated operational dataset maintained by the pipeline.

The pipeline does not simply overwrite the staging table.

Instead, it determines what has changed between the source and staging layers.

---

7. Preparing the Source Dataset

The first stage of the incremental process prepares the source data.

The source customer attributes are selected and a SHA-256 hash is generated.

Conceptually:

```
Source customer
      │
      ▼
Relevant attributes
      │
      ▼
Concatenated representation
      │
      ▼
SHA-256
      │
      ▼
row_hash
```

The resulting prepared source dataset is stored temporarily in:

```
#source_prepared
```

This temporary dataset becomes the comparison point for the incremental operations.

---

8. Hash-Based Change Detection

One of the central design decisions was the use of a row-level SHA-256 hash.

The hash is generated from customer attributes such as:

```
CustomerName
BirthDate
Gender
MaritalStatus
TotalChildren
HouseOwnerFlag
NumberCarsOwned
Education
Occupation
Continent
Country
State
City
YearlyIncome
DateFirstPurchase
```

The resulting value is stored as:

```
row_hash
```

The comparison logic is conceptually:

```
Source row hash
        │
        ├── same as staging hash
        │          ↓
        │      unchanged
        │
        └── different from staging hash
                   ↓
               changed
```

This provides a compact mechanism for detecting changes across multiple business attributes.

9. Inserting New Customers

The first incremental operation identifies source customers whose `CustomerID` does not exist in the staging table.

Conceptually:

```
Source
   │
   ├── Customer exists in staging → continue
   │
   └── Customer does not exist → INSERT
```

The number of inserted rows is captured immediately using:

```
@@ROWCOUNT
```

and stored in:

```
@rows_inserted
```

This value is later written into the pipeline log.

---

10. Updating Changed Customers

The second operation identifies existing customers whose data has changed.

The update condition checks:

```
staging.row_hash <> source.row_hash
```

or whether the record had previously been marked as deleted.

When a customer changes, the staging record is updated with the current source attributes and the new hash.

The deleted flag is also reset:

```
is_deleted = 0
```

This means that if a previously missing customer reappears in the source, the pipeline can restore the record to an active state.

The number of updated records is captured in:

```
@rows_updated
```

---

11. Soft Delete Handling

A particularly important part of the pipeline is the treatment of records that disappear from the source.

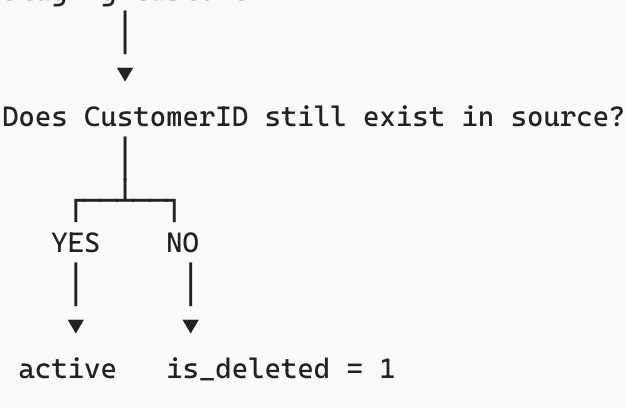
The pipeline does **not physically delete** the staging record.

Instead, it sets:

```
is_deleted = 1
```

This is known as a **soft delete**.

The logic is:



This approach preserves the record while allowing downstream processes to distinguish active and deleted customers.

It also provides a basic historical trace of records that were removed from the source.

---

12. Pipeline Logging

A dedicated table was created:

```
dbo.pipeline_log
```

The pipeline creates a new log entry at the beginning of each execution.

The initial status is:

```
running
```

A unique `run_id` is captured using `SCOPE_IDENTITY()`.

The run is therefore associated with a specific execution.

The log records:

```
run_id
pipeline_name
start_time
end_time
status
rows_inserted
rows_updated
rows_deleted
error_message
```

This creates an execution history rather than leaving the pipeline as a black box.

Evidence: Pipeline Log

	run_id	pipeline_name	start_time	end_time	status	rows_inserted	rows_updated	rows_deleted	error_message
2	16	customer_incremental_load	2026-08-17 21:36:10.6728429	2026-08-17 21:36:11.0168470	success	0	0	0	NULL
3	15	customer_incremental_load	2026-08-16 21:59:58.6960335	2026-08-16 21:59:58.9990332	success	0	0	0	NULL
4	14	customer_incremental_load	2026-08-16 17:16:57.2419528	2026-08-16 17:16:57.4729510	success	0	0	0	NULL
5	13	customer_incremental_load	2026-08-16 17:16:55.7318503	2026-08-16 17:16:55.9658474	success	0	0	0	NULL
6	12	customer_incremental_load	2026-08-16 17:14:07.4143721	2026-08-16 17:14:07.7263670	success	0	0	0	NULL
7	11	customer_incremental_load	2026-08-16 16:21:16.6116770	2026-08-16 16:21:16.9276295	success	0	0	0	NULL
8	10	customer_incremental_load	2026-08-16 15:29:14.9069908	2026-08-16 15:29:15.1599936	success	0	0	0	NULL
9	9	customer_incremental_load	2026-08-16 15:28:40.1997291	2026-08-16 15:28:40.4387265	failed	NULL	NULL	NULL	test transaction rollback
10	8	customer_incremental_load	2026-08-16 15:24:56.8720542	2026-08-16 15:24:57.1690088	success	0	0	0	NULL
11	7	customer_incremental_load	2026-08-16 15:24:15.3960114	2026-08-16 15:24:15.7000074	success	0	0	0	NULL
12	6	customer_incremental_load	2026-08-16 14:52:07.0459544	2026-08-16 14:52:07.3399517	success	0	0	0	NULL
13	5	customer_incremental_load	2026-08-16 14:51:07.3968333	2026-08-16 14:51:07.6947963	success	0	0	0	NULL

Caption: Pipeline execution log demonstrating execution tracking, status monitoring, row-count metrics, and error handling.

---

13. Transaction Management

The incremental operations are executed inside a SQL Server transaction.

The conceptual structure is:

```
BEGIN TRY
    BEGIN TRANSACTION

        Prepare source
        Insert new records
        Update changed records
        Soft-delete missing records
        Validate results

    COMMIT TRANSACTION

    Log success

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION
    Log failure
    THROW

END CATCH
```

This design is important because the pipeline consists of multiple dependent operations.

Without transaction handling, an error occurring halfway through the process could leave the staging table in a partially updated state.

With the transaction:

```
Success
   ↓
COMMIT
```

```
Failure
   ↓
ROLLBACK
```

This provides a much stronger reliability model.

---

14. Error Handling

The pipeline uses:

```
BEGIN TRY
...
END TRY

BEGIN CATCH
...
END CATCH
```

When an error occurs:

1. The transaction is rolled back if necessary.
2. The pipeline log is updated.
3. The status becomes `failed`.
4. The error message is stored.
5. The original error is re-raised using `THROW`.

During development, this error-handling design was actually tested.

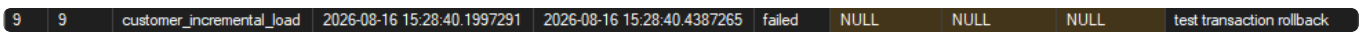
For example, an execution produced an error caused by an already existing temporary object:

```
There is already an object named '#source_prepared' in the database.
```

The failure was captured in the pipeline log.

This was useful because it demonstrated that the logging mechanism was not merely theoretical—it captured actual failed executions.

Evidence: Failed Pipeline Run



Caption: Example of pipeline error handling and failure logging during development.

---

15. Temporary Table Handling

The source preparation stage uses:

```
#source_prepared
```

as a temporary table.

During development, a repeated execution exposed an important practical issue: temporary objects can remain available within the same SQL session if they have not been removed.

The pipeline was therefore made more robust by explicitly using:

```
DROP TABLE IF EXISTS #source_prepared;
```

before recreating the temporary table.

This illustrates an important engineering lesson:

> A pipeline should be designed for repeated execution, not only for a single successful run.

---

16. Data Quality Validation

After the incremental operations, several validation queries are executed.

The first validation calculates:

```
total_rows
active_rows
deleted_rows
```

This gives a high-level view of the resulting staging population.

Duplicate Customer Validation

The pipeline checks for duplicate `CustomerID` values.

Conceptually:

```
CustomerID
    ↓
GROUP BY CustomerID
    ↓
HAVING COUNT(*) > 1
```

The expected result is no duplicate active/staging customer identifiers where uniqueness is required.

Active Records Missing from Source

The pipeline also checks whether an active staging customer is absent from the source.

This helps identify situations where the soft-delete process has failed.

The expected result is zero unexpected active records missing from the source.

Source Records Missing from Staging

A reverse validation checks whether a source customer is missing from staging or incorrectly marked as deleted.

This provides a second direction of reconciliation.

Together, these checks create a more complete validation strategy.

Evidence: SQL Validation Results

	total_rows	active_rows	deleted_rows
1	18485	18484	1

Caption: Data-quality validation performed after the incremental ETL process.

---

17. Incremental Pipeline Results

During the project, the pipeline was repeatedly executed and monitored through the logging table.

The resulting log demonstrated that successful runs could record:

```
status = success
rows_inserted
rows_updated
rows_deleted
```

The pipeline also demonstrated failed-run handling.

The final successful executions showed zero changes when no new source differences existed, which is the expected behavior for an incremental process when the source and staging layers are already synchronized.

This is an important characteristic of the pipeline:

> Re-running the pipeline without source changes does not continually insert or update the same records.

That demonstrates **idempotent-style incremental behavior** for the current synchronization logic.

---

18. Transition from SQL Server to PySpark

Once the SQL Server staging layer was functioning correctly, the project introduced PySpark.

The objective was not to recreate the SQL Server ETL in Spark.

Instead, the responsibilities were deliberately separated:

```
SQL Server
    ↓
Data ingestion and incremental maintenance

PySpark
    ↓
Analytical processing
```

This creates a clearer architecture and demonstrates different technologies performing different roles.

The active customer records were exported from:

```
staging.Customers
```

using:

```
WHERE is_deleted = 0
```

The technical `row_hash` and deletion flag were not required for the analytical dataset.

The resulting active customer dataset contained:

```
18,484 customers
16 columns
```

---

19. PySpark Environment

PySpark was installed and executed in Google Colab.

The Spark session was created using:

```
SparkSession.builder \
    .appName("CustomerDataEngineeringProject") \
    .getOrCreate()
```

The resulting Spark environment reported:

```
Spark Version: 4.0.4
Master: local[*]
Application: CustomerDataEngineeringProject
```

This demonstrates that Spark was running locally rather than on a paid cloud cluster.

Evidence: Spark Session

```
from pyspark.sql import SparkSession
```

```
spark = (SparkSession.builder.appName("CustomerDataEngineeringProject").getOrCreate())
```

```
spark
```

SparkSession - in-memory

SparkContext

Spark UI

Version
v4.0.4
Master
local[*]
AppName
CustomerDataEngineeringProject

Caption: Local PySpark environment used for analytical processing.

---

20. PySpark Data Ingestion

The exported active customer CSV was loaded into Spark using:

```
customers_spark_df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("customers_active.csv")
)
```

Schema inspection was then performed.

The resulting schema correctly identified fields such as:

```
CustomerID          integer
CustomerName        string
BirthDate           date
Gender              string
MaritalStatus       string
TotalChildren       integer
HouseOwnerFlag      integer
NumberCarsOwned     integer
Education           string
Occupation          string
Continent           string
Country             string
State               string
```

```
City                string
YearlyIncome        double
DateFirstPurchase   date
```

This schema inspection was important because data ingestion should always be validated rather than assumed to be correct.

During the first export attempt, the CSV was saved without column headers.

Spark consequently interpreted the first customer's values as column names.

For example, values such as:

```
11000
Jon Yang
1971-10-06
```

appeared as field names.

The issue was identified by inspecting the Spark schema, the CSV was corrected to include column headers, and the data was successfully ingested afterward.

This was a useful example of real-world data-engineering debugging rather than simply executing a perfect pipeline from the beginning.

Evidence: Correct Spark Schema

```
customers_spark_df.printSchema()
```

```
root
 |-- CustomerID: integer (nullable = true)
 |-- CustomerName: string (nullable = true)
 |-- BirthDate: date (nullable = true)
 |-- Gender: string (nullable = true)
 |-- MaritalStatus: string (nullable = true)
 |-- TotalChildren: integer (nullable = true)
 |-- HouseOwnerFlag: integer (nullable = true)
 |-- NumberCarsOwned: integer (nullable = true)
 |-- Education: string (nullable = true)
 |-- Occupation: string (nullable = true)
 |-- Continent: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- State: string (nullable = true)
 |-- City: string (nullable = true)
 |-- YearlyIncome: double (nullable = true)
 |-- DateFirstPurchase: date (nullable = true)
```

Caption: Correct schema inferred by Spark after the CSV header issue was resolved.

---

21. Spark DataFrame Processing

The imported dataset was converted into the main working DataFrame:

```
customers_df
```

The dataset was checked using:

```
customers_df.count()
```

and:

```
customers_df.show(5)
```

The resulting dataset contained:

```
18,484 rows
16 columns
```

This established that the active customer population exported from SQL Server had successfully reached the Spark processing layer.

---

22. Country-Level Customer Analysis

The first major analytical transformation grouped customers by country.

The calculation produced:

- customer count,
- average yearly income.

The resulting output was:

Country	Customer Count	Average Yearly Income
United States	7,819	63,616.83
Australia	3,591	64,338.62
United Kingdom	1,913	52,169.37
France	1,810	35,762.43
Germany	1,780	42,943.82
Canada	1,571	57,167.41

The United States represented the largest customer population in the dataset.

Australia had the highest average yearly income among the six countries.

This demonstrates that the Spark layer can convert the operational customer dataset into a useful analytical view.

Evidence: Country Analysis

```
+--------------+-------------+-------------------+
|Country       |CustomerCount|AverageYearlyIncome|
+--------------+-------------+-------------------+
|United States |7819         |63616.83           |
|Australia     |3591         |64338.62           |
|United Kingdom|1913         |52169.37           |
|France        |1810         |35762.43           |
|Germany       |1780         |42943.82           |
|Canada        |1571         |57167.41           |
+--------------+-------------+-------------------+
```

Caption: Country-level customer counts and average yearly income calculated using PySpark.

---

23. Income Segmentation

The project also created a derived analytical field:

```
IncomeSegment
```

Customers with yearly income greater than or equal to $100,000 were classified as:

```
High Income
```

Other customers were classified as:

```
Standard Income
```

The resulting segmentation was:

Income Segment	Customer Count	Average Yearly Income
Standard Income	16,286	48,757.83
High Income	2,198	120,641.49

Therefore:

```
Standard Income: 16,286 / 18,484 ≈ 88.1%
High Income:      2,198 / 18,484 ≈ 11.9%
```

This demonstrates how PySpark can be used to derive analytical attributes from operational data.

Evidence: Income Segmentation

```
+----------+-----------------+------------+---------------+
|CustomerID|     CustomerName|YearlyIncome|  IncomeSegment|
+----------+-----------------+------------+---------------+
|     11000|         Jon Yang|     90000.0|Standard Income|
|     11001|     Eugene Huang|     60000.0|Standard Income|
|     11002|     Ruben Torres|     60000.0|Standard Income|
|     11003|      Christy Zhu|     70000.0|Standard Income|
|     11004|Elizabeth Johnson|     80000.0|Standard Income|
|     11005|       Julio Ruiz|     70000.0|Standard Income|
|     11006|    Janet Alvarez|     70000.0|Standard Income|
|     11007|      Marco Mehta|     60000.0|Standard Income|
|     11008|      Rob Verhoff|     60000.0|Standard Income|
|     11009|  Shannon Carlson|     70000.0|Standard Income|
+----------+-----------------+------------+---------------+
+---------------+-------------+-------------------+
|  IncomeSegment|CustomerCount|AverageYearlyIncome|
+---------------+-------------+-------------------+
|Standard Income|        16286|           48757.83|
|    High Income|         2198|          120641.49|
+---------------+-------------+-------------------+
```

Caption: Customer segmentation based on yearly income.

---

24. Spark Transformations and Actions

The project deliberately demonstrated the distinction between **transformations** and **actions**.

Examples of transformations included:

```
filter()
select()
withColumn()
groupBy()
agg()
orderBy()
```

Examples of actions included:

```
show()
count()
```

This distinction is important because Spark uses **lazy evaluation**.

A transformation does not necessarily execute immediately.

Instead, Spark builds a logical and physical execution plan.

The plan is evaluated when an action requires the result.

---

25. Spark Physical Execution Plan

The execution plan was inspected using:

```
customer_segments_df.explain()
```

The resulting physical plan included operations such as:

```
FileScan csv
Project
CASE WHEN
```

This demonstrated that Spark was constructing an execution plan rather than simply executing Python statements row by row.

This is an important conceptual difference between ordinary Python data processing and Spark's execution model.

Evidence: Spark Physical Plan

== Physical Plan ==
*(1) Project [CustomerID#17, CustomerName#18, BirthDate#19, Gender#20, MaritalStatus#21, TotalChildren#22, HouseOwnerFlag#23, NumberCarsOwned#24, Education#25, Occupation#26, Continent#27, Country#28, State#29, City#30, YearlyIncome#31, DateFirstPurchase#32, CASE WHEN (YearlyIncome#31 >= 100000.0) THEN High Income ELSE Standard Income END AS IncomeSegment#226]
+- FileScan csv [CustomerID#17,CustomerName#18,BirthDate#19,Gender#20,MaritalStatus#21,TotalChildren#22,HouseOwnerFlag#23,NumberCarsOwned#24,Education#25,Occupation#26,Continent#27,Country#28,State#29,City#30,YearlyIncome#31,DateFirstPurchase#32] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/content/customers_active.csv], PartitionFilters: [], PushedFilters: [], ReadSchema:

struct<CustomerID:int,CustomerName:string,BirthDate:date,Gender:string,MaritalStatus:string,Total...

Caption: Spark physical execution plan demonstrating lazy evaluation and query planning.

---

26. Parquet Storage

After ingesting the CSV, the customer dataset was written to Parquet.

Conceptually:

```
CSV
 ↓
PySpark DataFrame
 ↓
Parquet
```

The Parquet format was selected because it is well suited to analytical workloads and preserves data types more effectively than a simple CSV representation.

The resulting Parquet dataset was then read back into Spark.

This created an additional validation point:

```
Write Parquet
      ↓
Read Parquet
      ↓
Inspect schema and data
```

The data successfully survived this storage conversion.

Evidence: Parquet Output

```
customers_df.write \
    .mode("overwrite") \
    .parquet("customers_parquet")
```

```
customers_parquet_df = spark.read.parquet("customers_parquet")

customers_parquet_df.printSchema()
customers_parquet_df.show(5)
```

```
root
 |-- CustomerID: integer (nullable = true)
 |-- CustomerName: string (nullable = true)
 |-- BirthDate: date (nullable = true)
 |-- Gender: string (nullable = true)
 |-- MaritalStatus: string (nullable = true)
 |-- TotalChildren: integer (nullable = true)
 |-- HouseOwnerFlag: integer (nullable = true)
 |-- NumberCarsOwned: integer (nullable = true)
 |-- Education: string (nullable = true)
 |-- Occupation: string (nullable = true)
 |-- Continent: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- State: string (nullable = true)
 |-- City: string (nullable = true)
 |-- YearlyIncome: double (nullable = true)
 |-- DateFirstPurchase: date (nullable = true)
```

==========

```
+----------+----------------+----------+------+-------------+-------------+--------------+---------------+---------+------------+---------+---------+
|CustomerID|    CustomerName| BirthDate|Gender|MaritalStatus|TotalChildren|HouseOwnerFlag|NumberCarsOwned|Education|  Occupation|Continent|  Country|
+----------+----------------+----------+------+-------------+-------------+--------------+---------------+---------+------------+---------+---------+
|     11000|        Jon Yang|1971-10-06|     M|            M|            2|             1|              0|Bachelors|Professional|  Pacific|Australia|
|     11001|    Eugene Huang|1976-05-10|     M|            S|            3|             0|              1|Bachelors|Professional|  Pacific|Australia|
|     11002|    Ruben Torres|1971-02-09|     M|            M|            3|             1|              1|Bachelors|Professional|  Pacific|Australia|
|     11003|     Christy Zhu|1973-08-14|     F|            S|            0|             0|              1|Bachelors|Professional|  Pacific|Australia|
|     11004|Elizabeth Johnson|1979-08-05|     F|            S|            5|             1|              4|Bachelors|Professional|  Pacific|Australia|
+----------+----------------+----------+------+-------------+-------------+--------------+---------------+---------+------------+---------+---------+
only showing top 5 rows
```

Caption: Customer data converted from CSV to Parquet and successfully read back using Spark.

---

27. Partitioned Parquet Dataset

A second Parquet dataset was created using:

```
.partitionBy("Country")
```

This created a physical directory structure similar to:

```
customers_by_country/
    Country=Australia/
    Country=Canada/
    Country=France/
    Country=Germany/
    Country=United Kingdom/
    Country=United States/
```

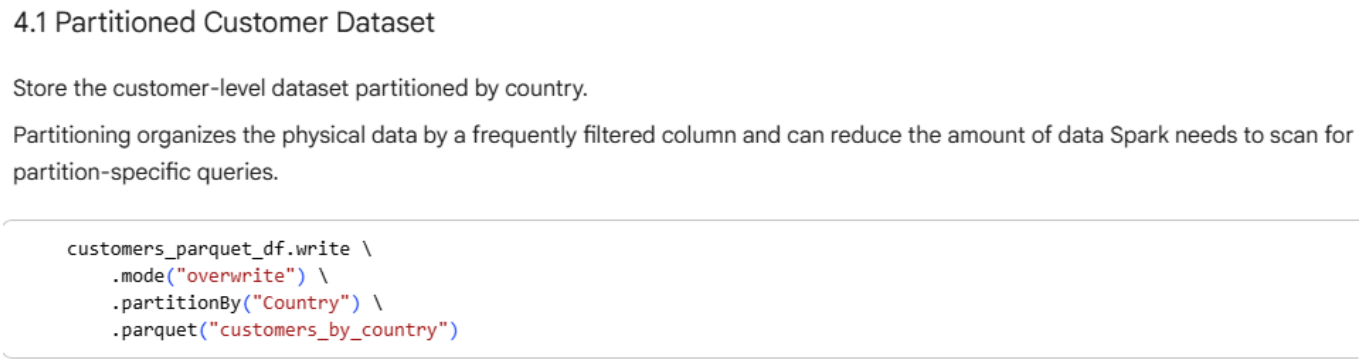
The important point is that partitioning was applied to the **customer-level dataset**, not to the already aggregated country-level result.

This distinction matters.

Partitioning the customer-level dataset provides a meaningful physical organization because thousands of customer records exist within each country.

Partitioning a six-row country-level summary would provide little benefit and could result in many tiny output partitions.

Evidence: Partitioned Dataset



Caption: Customer-level Parquet dataset physically partitioned by country.

---

28. Partition Pruning

The project then demonstrated why partitioning can be useful.

The partitioned dataset was read and filtered to:

```
Country = Australia
```

The physical execution plan contained:

```
PartitionFilters:
[isnotnull(Country#494), (Country#494 = Australia)]
```

This is significant evidence.

Spark recognized that `Country` was a partition column and could use the partition filter during the scan.

Conceptually:

```
customers_by_country/
        │
        ├── Australia
        ├── Canada
        ├── France
        ├── Germany
        ├── United Kingdom
        └── United States
                │
                ▼
       filter Country = Australia
                │
                ▼
        Australia partition
```

This behavior is called **partition pruning**.

The project therefore demonstrated not merely that partitioning exists, but also how Spark can take advantage of partitioned storage for selective queries.

Evidence: Partition Pruning

== Physical Plan ==
*(1) ColumnarToRow
+- FileScan parquet
[CustomerID#408,CustomerName#409,BirthDate#410,Gender#411,MaritalStatus#412,TotalChildren#413,HouseOwnerFlag#414,NumberCarsOwned#415,Education#416,Occupation#417,Continent#418,State#419,City#420,YearlyIncome#421,DateFirstPurchase#422,Country#423]
Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths)

[file:/content/customers_by_country], PartitionFilters: [isnotnull(Country#423), (Country#423 = Australia)], PushedFilters: [], ReadSchema: struct<CustomerID:int,CustomerName:string,BirthDate:date,Gender:string,MaritalStatus:string, Total...

Caption: Spark execution plan demonstrating partition pruning for a country-specific query.

---

29. Important Distinction: Grouping vs Partitioning

One of the useful lessons from the project was that:

```
groupBy("Country")
```

and:

```
partitionBy("Country")
```

are not the same operation.

`groupBy("Country")` is an analytical transformation.

It means:

> Calculate something separately for each country.

`partitionBy("Country")` is a physical storage decision.

It means:

> Organize the stored data into country-specific partitions.

Partition pruning occurs when a query can use that physical organization to avoid scanning irrelevant partitions.

For example:

```
groupBy("Country")
```

requires statistics for every country, so all country partitions are relevant.

However:

```
filter(Country == "Australia")
```

can allow Spark to avoid reading the other country partitions.

Understanding this distinction was an important part of the PySpark learning process.

---

30. Final Analytical Dataset

The final analytical dataset calculates country-level metrics including:

```
Country
CustomerCount
AverageYearlyIncome
AverageChildren
AverageCarsOwned
```

These metrics provide a compact analytical representation of the customer population.

The final result is stored as:

```
final_customer_country_metrics/
```

in Parquet format.

This represents the final analytical output of the Spark component.

Evidence: Final Analytical Dataset

```
country_metrics_df.show(truncate=False)
```

```
+--------------+-------------+-------------------+---------------+-----------------+
|Country       |CustomerCount|AverageYearlyIncome|AverageChildren|AverageCarsOwned|
+--------------+-------------+-------------------+---------------+-----------------+
|United States |7819         |63616.83           |2.0            |1.52             |
|Australia     |3591         |64338.62           |1.63           |1.91             |
|United Kingdom|1913         |52169.37           |1.63           |1.22             |
|France        |1810         |35762.43           |1.74           |1.25             |
|Germany       |1780         |42943.82           |1.67           |1.21             |
|Canada        |1571         |57167.41           |2.12           |1.46             |
+--------------+-------------+-------------------+---------------+-----------------+
```

Caption: Final country-level analytical dataset generated using PySpark.

31. Data Quality Validation in PySpark

The final Spark output was also validated.

The number of countries was checked.

Required fields were checked for null values.

Most importantly, the total customer count was reconciled.

The country-level customer counts summed to:

```
18,484
```

This matched the active customer population exported from SQL Server.

Therefore:

```
SQL Server active customers
          =
PySpark analytical customer population
          =
18,484
```

This reconciliation is an important piece of evidence because it demonstrates that the Spark transformation did not unintentionally lose or duplicate customer records during the analytical process.

Evidence: Reconciliation

```
print(f"Number of countries: {country_metrics_df.count()}")

country_metrics_df.show(truncate=False)
```

```
Number of countries: 6
+--------------+-------------+-------------------+---------------+----------------+
|Country       |CustomerCount|AverageYearlyIncome|AverageChildren|AverageCarsOwned|
+--------------+-------------+-------------------+---------------+----------------+
|United States |7819         |63616.83           |2.0            |1.52            |
|Australia     |3591         |64338.62           |1.63           |1.91            |
|United Kingdom|1913         |52169.37           |1.63           |1.22            |
|France        |1810         |35762.43           |1.74           |1.25            |
|Germany       |1780         |42943.82           |1.67           |1.21            |
|Canada        |1571         |57167.41           |2.12           |1.46            |
+--------------+-------------+-------------------+---------------+----------------+
```

==========

```
country_metrics_df.select(
    "Country",
    "CustomerCount",
    "AverageYearlyIncome",
    "AverageChildren",
    "AverageCarsOwned"
).where(
    col("Country").isNull()
).show()
```

```
+-------+-------------+-------------------+---------------+----------------+
|Country|CustomerCount|AverageYearlyIncome|AverageChildren|AverageCarsOwned|
+-------+-------------+-------------------+---------------+----------------+
+-------+-------------+-------------------+---------------+----------------+
```

==========

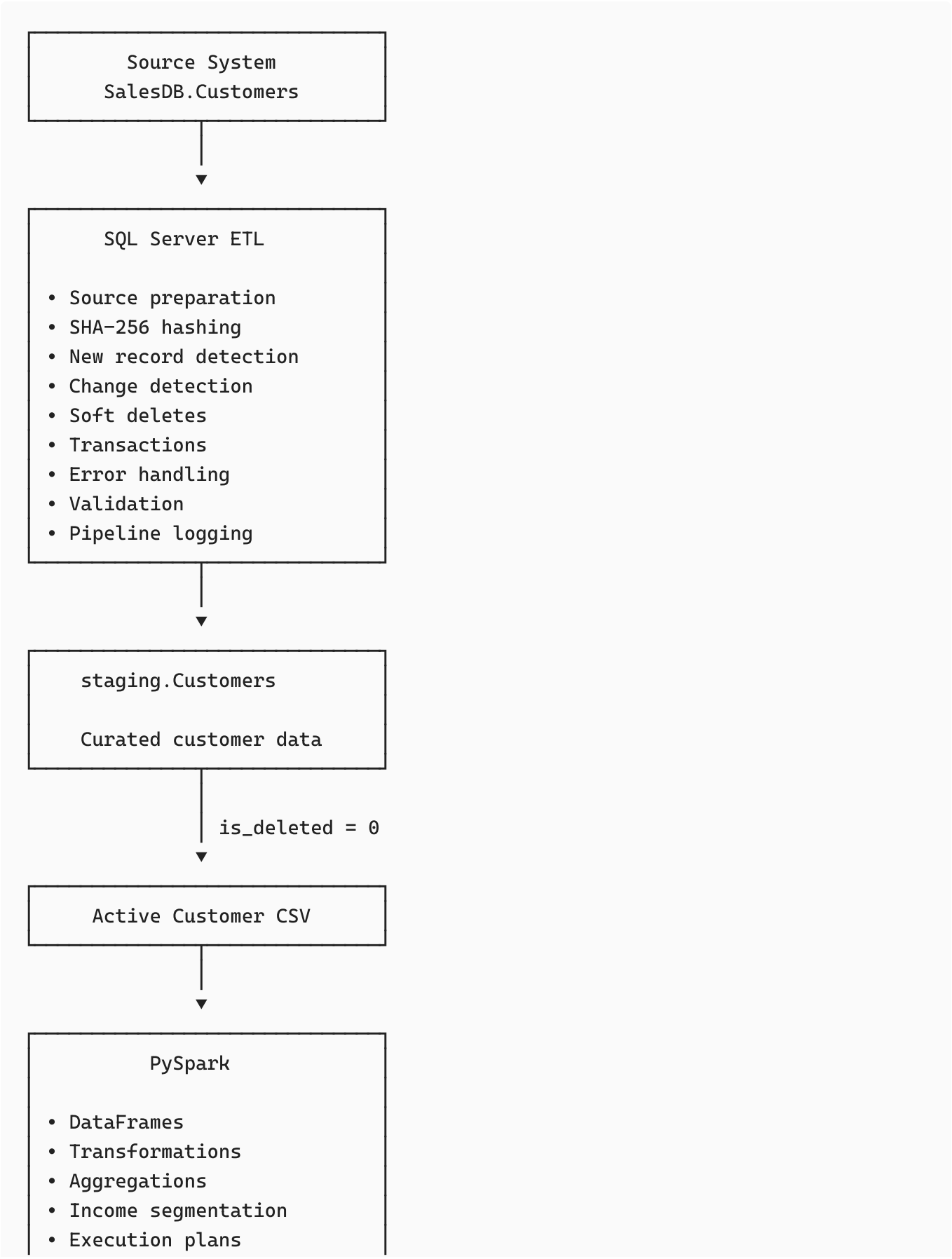
```
from pyspark.sql.functions import sum as spark_sum

country_metrics_df.select(
    spark_sum("CustomerCount").alias("TotalCustomers")
).show()
```

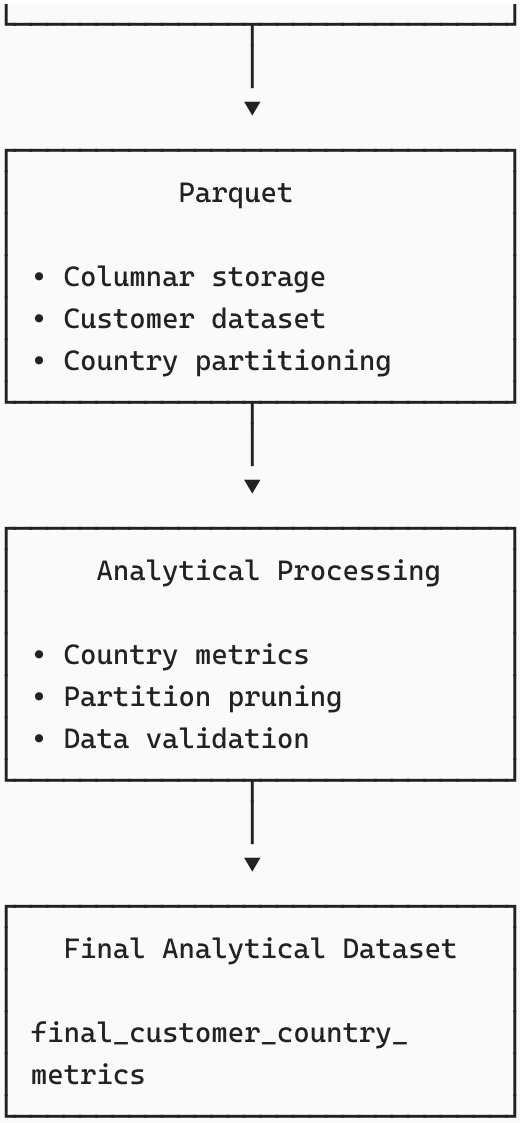
```
+--------------+
|TotalCustomers|
+--------------+
|         18484|
+--------------+
```

Caption: Reconciliation of the PySpark analytical population against the SQL Server active customer population.

32. Complete End-to-End Architecture

The completed project can now be represented as follows:





---

33. Evidence of Engineering Reliability

One of the strongest aspects of this project is that reliability was treated as part of the pipeline rather than an afterthought.

The project includes evidence for:

Successful executions

```
running → success
```

with:

- run ID,

- timestamps,
- row counts.

Failed executions

```
running → failed
```

with:

- error message,
- timestamp,
- failed status.

Recovery

Transactions allow failed data modifications to be rolled back.

Repeated execution

Running the pipeline again without source changes results in no unnecessary inserts or updates.

Validation

The pipeline checks the resulting data rather than assuming that successful execution means correct data.

---

34. Key Technical Challenges Encountered

The project was not completed by simply writing code once and receiving the expected output.

Several practical issues were encountered and resolved.

34.1 Pipeline Log Variable Scope

An early version of the SQL script attempted to use row-count variables that had not been declared in the relevant execution scope.

This produced:

```
Must declare the scalar variable "@rows_inserted".
```

The script was corrected by explicitly declaring:

```
@rows_inserted
@rows_updated
@rows_deleted
```

before the pipeline operations.

This reinforced the importance of variable scope and complete execution context in T-SQL.

34.2 Temporary Table Collision

Repeated execution resulted in:

```
There is already an object named '#source_prepared'
```

The issue was addressed using:

```
DROP TABLE IF EXISTS #source_prepared;
```

before creating the temporary table.

This made the script safer for repeated development and testing.

34.3 Pipeline Error Logging

The error was intentionally observed in the pipeline log rather than simply ignored.

This verified that the `TRY/CATCH` and logging mechanism was functioning.

34.4 CSV Header Problem

During the first Spark ingestion attempt, the CSV did not contain column headers.

Spark consequently interpreted the first customer's values as column names.

The problem was identified immediately through:

```
printSchema()
```

The CSV was corrected and reloaded.

This demonstrated the importance of inspecting incoming schemas rather than trusting the input format.

34.5 Python `sum()` vs Spark `sum()`

During PySpark validation, the following produced an error:

```
sum("CustomerCount")
```

because Python's built-in `sum()` function was being used instead of Spark's aggregation function.

The problem was corrected using:

```
from pyspark.sql.functions import sum as spark_sum
```

and:

```
spark_sum("CustomerCount")
```

This was a small but useful example of namespace collisions when working with Python and Spark APIs.

---

35. What I Learned

This project provided learning in several different areas of data engineering.

35.1 Incremental ETL Is More Than INSERT Statements

A reliable incremental pipeline needs to understand the difference between:

```
New
Changed
```

```
Deleted
Unchanged
```

The project implemented each of these states.

35.2 Data Pipelines Need Observability

A pipeline that says nothing about what happened is difficult to maintain.

The logging table introduced:

```
run_id
status
timestamps
row counts
errors
```

This created basic pipeline observability.

35.3 Transactions Protect Data Integrity

The project demonstrated why multiple dependent transformations should be treated as one logical unit.

If the process fails:

```
ROLLBACK
```

protects the staging layer from being left partially updated.

35.4 Validation Is a Core Engineering Step

The project taught that validation should occur at multiple levels:

```
Source validation
       ↓
ETL validation
       ↓
```

```
Staging validation
       ↓
Spark ingestion validation
       ↓
Analytical output validation
```

The final reconciliation of 18,484 customers is particularly important because it connects the SQL Server and PySpark layers.

35.5 Spark Is Not Just "Python With More Data"

The PySpark work demonstrated concepts that are fundamental to Spark:

- lazy evaluation,
- transformations,
- actions,
- execution plans,
- columnar storage,
- partitioning,
- partition pruning.

The `explain()` output helped make Spark's execution model visible instead of treating Spark as a black box.

35.6 Storage Design Matters

The project demonstrated that simply saying:

> "I used Parquet."

is not enough.

The way data is physically organized matters.

Partitioning can be useful for selective queries, while unnecessary partitioning can create many small files with little benefit.

Understanding when **not** to partition is just as important as knowing how to partition.

---

36. Skills Demonstrated

This project demonstrates practical experience in the following areas.

SQL / Database Engineering

- T-SQL
- SQL Server
- joins
- temporary tables
- aggregations
- updates
- conditional logic
- hashing
- `ROWCOUNT`
- `SCOPE_IDENTITY`
- transactions
- `TRY/CATCH`
- error handling
- soft deletes

ETL / Data Engineering

- incremental loading
- change detection
- source-to-target synchronization
- data validation
- pipeline logging
- auditability
- idempotent-style processing
- failure recovery
- data-quality checks

PySpark

- SparkSession
- Spark DataFrames
- schema inference
- `select`
- `filter`

- `withColumn`
- `groupBy`
- `agg`
- `orderBy`
- lazy evaluation
- physical execution plans
- Parquet
- partitioning
- partition pruning

Engineering Practices

- debugging
- iterative development
- validation
- reproducibility
- documentation
- version control
- GitHub project organization

---

37. Project Outcomes

The project successfully produced a complete local data engineering workflow.

The SQL Server component maintains a curated customer staging layer through incremental processing.

The pipeline is capable of identifying:

```
New customers
Changed customers
Deleted customers
Unchanged customers
```

The pipeline also records its execution history and failures.

The PySpark component successfully processed the active customer population of:

```
18,484 customers
```

The Spark layer generated analytical insights including:

```
United States
7,819 customers
Average income: $63,616.83

Australia
3,591 customers
Average income: $64,338.62

United Kingdom
1,913 customers
Average income: $52,169.37

France
1,810 customers
Average income: $35,762.43

Germany
1,780 customers
Average income: $42,943.82

Canada
1,571 customers
Average income: $57,167.41
```

The income segmentation produced:

```
Standard Income
16,286 customers
Average income: $48,757.83

High Income
2,198 customers
Average income: $120,641.49
```

The final analytical customer count reconciled to:

```
18,484
```

This confirmed consistency between the SQL Server active population and the PySpark analytical population.

---

38. Additional Evidence and Screenshots

Evidence 1 — SQL Server Source Data

	CustomerID	CustomerName	BirthDate	Gender	MaritalStatus	TotalChildren	HouseOwnerFlag	NumberCarsOwned	Education	Occupation	Continent	Country	State	City	YearlyIncome	DateFirstPurchase
1	11000	Jon Yang	1971-10-06	M	M	2	1	0	Bachelors	Professional	Pacific	Australia	Queensland	Rockhampton	90000.00	2011-01-19
2	11001	Eugene Huang	1976-05-10	M	S	3	0	1	Bachelors	Professional	Pacific	Australia	Victoria	Seaford	60000.00	2011-01-15
3	11002	Ruben Torres	1971-02-09	M	M	3	1	1	Bachelors	Professional	Pacific	Australia	Tasmania	Hobart	60000.00	2011-01-07
4	11003	Christy Zhu	1973-08-14	F	S	0	0	1	Bachelors	Professional	Pacific	Australia	New Sout...	North Ryde	70000.00	2010-12-29
5	11004	Elizabeth Joh...	1979-08-05	F	S	5	1	4	Bachelors	Professional	Pacific	Australia	New Sout...	Wollongong	80000.00	2011-01-23
6	11005	Julio Ruiz	1976-08-01	M	S	0	1	1	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	70000.00	2010-12-30
7	11006	Janet Alvarez	1976-12-02	F	S	0	1	1	Bachelors	Professional	Pacific	Australia	New Sout...	Matraville	70000.00	2011-01-24
8	11007	Marco Mehta	1969-11-06	M	M	3	1	2	Bachelors	Professional	Pacific	Australia	Victoria	Warrnambool	60000.00	2011-01-09
9	11008	Rob Verhoff	1975-07-04	F	S	4	1	3	Bachelors	Professional	Pacific	Australia	Victoria	Bendigo	60000.00	2011-01-25
10	11009	Shannon Carl...	1969-09-29	M	S	0	0	1	Bachelors	Professional	Pacific	Australia	Queensland	Hervey Bay	70000.00	2011-01-27
11	11010	Jacquelyn Su...	1969-08-05	F	S	0	0	1	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	70000.00	2011-01-14
12	11011	Curtis Lu	1969-05-03	M	M	4	1	4	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	60000.00	2010-12-30
13	11012	Lauren Walker	1979-01-14	F	M	2	1	2	Bachelors	Managem...	North A...	United...	Washington	Bremerton	100000.00	2013-03-16
14	11013	Ian Jenkins	1979-08-03	M	M	2	1	3	Bachelors	Managem...	North A...	United...	Oregon	Lebanon	100000.00	2013-04-13
15	11014	Sydney Benn...	1973-11-06	F	S	3	0	3	Bachelors	Managem...	North A...	United...	Washington	Redmond	100000.00	2013-03-23
16	11015	Chloe Young	1984-08-26	F	S	0	0	1	Partial C...	Skilled Ma...	North A...	United...	California	Burbank	30000.00	2013-01-18
17	11016	Wyatt Hill	1984-10-25	M	M	0	1	1	Partial C...	Skilled Ma...	North A...	United...	California	Imperial Bea...	30000.00	2013-02-09
18	11017	Shannon Wa...	1949-12-24	F	S	4	1	2	High Sc...	Skilled Ma...	Pacific	Australia	Victoria	Sunbury	20000.00	2011-01-12

Shows the source customer dataset and its relevant fields.

Evidence 2 — Staging Table

NumberCarsOwned	Education	Occupation	Continent	Country	State	City	YearlyIncome	DateFirstPurchase	row_hash	is_deleted
0	Bachelors	Professional	Pacific	Iceland	Queensland	Rockhampton	90000.00	2011-01-19	3E679923ACA81167C768E8CA4CAC1DF971CE6A25D6679C95BE4D7EF1735EB8A4	0
1	Bachelors	Professional	Pacific	Australia	Victoria	Seaford	60000.00	2011-01-15	CA3E8C94600FAFE66EA960CE0513D70F2429B39FB6B870130BED6176160FDAE7	0
1	Bachelors	Professional	Pacific	Australia	Tasmania	Hobart	60000.00	2011-01-07	1338D5EE8A63D508A2756A5772A54948ABC8B08F35420102969955E87DE40201F	0
1	Bachelors	Professional	Pacific	Australia	New Sout...	North Ryde	70000.00	2010-12-29	3D0D68354CE9E88ED9F99D984133E887ED915887002E707B545DB11A921BF07C	0
4	Bachelors	Professional	Pacific	Australia	New Sout...	Wollongong	80000.00	2011-01-23	5EFE97849D4CC65C52626BD3BBF07221006B5FFD0E46DD1274DB8491EF2971CE	0
1	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	70000.00	2010-12-30	12187FC8F6A8EE732669E1D40FED252DDA92F996D137498124BB2E3963BEB207	0
1	Bachelors	Professional	Pacific	Australia	New Sout...	Matraville	70000.00	2011-01-24	CDE471594200612A060BA31A8BCA74D9E60F2562B65998CB42E7181127C4A44E	0
2	Bachelors	Professional	Pacific	Australia	Victoria	Warrnambool	60000.00	2011-01-09	3D848672B2E700C97EB7E36639F8FD19096BE159D41C697E482858D28F755A35	0
3	Bachelors	Professional	Pacific	Australia	Victoria	Bendigo	60000.00	2011-01-25	091516CFB568E26945CEC622C85FF2E82A71929C3C74657AB1688BA2FFEB3134	0
1	Bachelors	Professional	Pacific	Australia	Queensland	Hervey Bay	70000.00	2011-01-27	423CA9B2CFD8F13946BD21322F44120E6BD97BD69B2E7020F69727E8A35F6BD0	0
1	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	70000.00	2011-01-14	5D658A5FC91AB2140FE9A304CB719704E7C5DB6F304435F121C60A63D52FEBB0	0
4	Bachelors	Professional	Pacific	Australia	Queensland	East Brisbane	60000.00	2010-12-30	65A8810EEA3F0B43A9431BBE6C9D3FFFE6A1093D6832DCB1D3B282F5160B1B12	0
2	Bachelors	Managem...	North A...	United...	Washington	Bremerton	100000.00	2013-03-16	506FD49A33F1BCE5E93EBE66B699A849CEE1B793FAE888D67071ED1C67AE9451	0
3	Bachelors	Managem...	North A...	United...	Oregon	Lebanon	100000.00	2013-04-13	ED8C3F19F1C19AC80BD1B81035B3CB433CF0156B93DB3FD816175F50E5F98E6B	0
3	Bachelors	Managem...	North A...	United...	Washington	Redmond	100000.00	2013-03-23	940EB23A5DC1888AFBFCAAAA3FD74807D9DF907FE4DB14D54308E967286A791C	0
1	Partial C...	Skilled Ma...	North A...	United...	California	Burbank	30000.00	2013-01-18	6742B227B3B83470C4091EC36609CA5D5A1A6CAC3C72218C16E24432BB1837F3	0
1	Partial C...	Skilled Ma...	North A...	United...	California	Imperial Bea...	30000.00	2013-02-09	A0C1AF1993C4EE6A961DF5AB7E13CBAC34AF72EAFA3C327AC97C59FE953FB35D	0
2	High Sc...	Skilled Ma...	Pacific	Australia	Victoria	Sunbury	20000.00	2011-01-12	D21910354A4017F6B0A2F8316C449B4D5B0E12A8DF1B05DB17CB386BC289988D	0
2	Partial C...	Clerical	Pacific	Australia	Victoria	Bendigo	30000.00	2011-01-17	A3E09064329AFC0F8727E241A71621BCDB2363CE751C3DC22CF86EEBA94C6A4F	0
2	High Sc...	Skilled Ma...	North A...	Canada	British Col...	Langley	40000.00	2013-02-12	BCCD525D825AA28A2650CB02112FB77EAEFD2E87E3D13CC32AE6077DE34A3416	0
2	High Sc...	Skilled Ma...	North A...	Canada	British Col...	Metchosin	40000.00	2012-12-29	2D52EE84F6D5DFC3FA3560A936E94A8BC5AB9032C3BABF442A54000602F40D9E	0

Shows the staging table including the incremental-processing fields such as `row_hash` and `is_deleted`.

---

39. Repository Structure

The project is maintained in a dedicated GitHub repository:

```
customer-data-engineering-pipeline
```

The repository contains the project's SQL scripts, datasets, PySpark notebook and documentation.

A logical structure for the repository is:

```
customer-data-engineering-pipeline/
│
├── SQL scripts
│
├── raw_source_customers_table.csv
│
├── customers_active.csv
│
├── PySpark notebook
│
└── Project report
```

The repository is intended to provide a reproducible record of the project and its development.

The SQL scripts demonstrate the database engineering layer, while the PySpark notebook demonstrates the analytical processing layer.

---

40. Final Reflection

The most important outcome of this project was not simply producing a working SQL script or a Spark notebook.

The project demonstrated how different data engineering components can work together.

The SQL Server layer answers:

> **How do I reliably maintain a current and auditable customer dataset?**

The PySpark layer answers:

> **How can I take that curated dataset and process it for analytical purposes?**

The validation layer answers:

> **How do I know that the pipeline produced a correct result?**

The logging layer answers:

> **How do I know what happened during each pipeline execution?**

The Parquet and partitioning work answers:

> **How should analytical data be stored and accessed efficiently?**

Together, these concepts form a much more realistic representation of a data engineering workflow than a simple script that reads a CSV and calculates an average.

The project also demonstrated that cloud infrastructure is not a prerequisite for learning or demonstrating these concepts. By using SQL Server locally and PySpark in local mode, it was possible to build and demonstrate a meaningful end-to-end pipeline while keeping the environment accessible and avoiding unnecessary paid cloud infrastructure.

---

41. Conclusion

This project successfully demonstrates an end-to-end customer data engineering workflow built around **SQL Server and PySpark**.

The SQL Server layer implements incremental ETL with:

- hash-based change detection,
- new-record insertion,
- changed-record updates,
- soft deletes,
- transaction management,
- rollback handling,
- pipeline logging,
- error handling,
- and data-quality validation.

The PySpark layer extends the pipeline into analytical processing through:

- Spark DataFrames,
- transformations,
- aggregations,
- income segmentation,
- lazy evaluation,
- physical execution plans,
- Parquet storage,
- partitioning,
- partition pruning,

- and final analytical outputs.

The final active customer population of **18,484 records** was successfully reconciled between the SQL Server and PySpark layers.

Most importantly, the project demonstrates an understanding of the **reasoning behind the technologies**, not simply their syntax.

The overall engineering approach can be summarized as:

```
Reliable ingestion
        ↓
Incremental processing
        ↓
Data quality
        ↓
Auditability
        ↓
Curated data
        ↓
Analytical processing
        ↓
Efficient storage
        ↓
Validated analytical output
```

This project therefore represents a practical demonstration of the core principles of modern data engineering: **building pipelines that are incremental, reliable, observable, validated, and useful for downstream analytics.**